

Predict Once, Query Many: World Models that Forecast Conditional Trajectory Abstractions

Anonymous CVPR submission

Paper ID *****

Abstract

World models for visual control usually predict the future frame by frame and leave it to a planner to extract what matters for a decision. This is general but costly: prediction cost grows with the horizon and the number of tokens per frame, errors compound over the rollout, and most of each predicted frame restates what the observed history already shows. At the other extreme, a direct scorer maps history, action, and goal to a score; it is cheap but entangles dynamics with the goal and must generalize anew to every goal it was not trained on. We ask whether a world model can instead predict a compact abstraction of an entire future segment that remains sufficient for decision making. We propose Conditional Trajectory Abstraction (CTA). During training only, a source encoder compresses the actual future segment of an action chunk into a few discrete tokens conditioned on the observed history, so that the code spends its capacity on what the action changes. A reader that sees the same history answers self-supervised visual-temporal queries from the code, such as progress toward a goal image or the order of two events, without ever seeing the action. An action-conditioned world model predicts the code autoregressively from the history and a candidate chunk, and the code is co-designed for query retention, rate, and predictability from actions. In policy-guided planning, a frozen policy proposes candidate chunks, the world model predicts one code per candidate, and the reader scores it against any goal without further prediction. No component uses rewards or success labels. On PushT and LIBERO-Goal, CTA [main result placeholder].

1. Introduction

A world model lets an agent evaluate an action before taking it. The dominant recipe for visual control predicts future observations, in pixels, discrete tokens, or the features of a pretrained encoder, one frame after another, and a planner then scores the imagined frames against a goal [3, 17, 28, 38].

Predicting frames keeps the model general, because any question about the future can in principle be answered from them. It also makes the model expensive in exactly the place where planning is most demanding. The cost of evaluating one candidate grows with the horizon and with the number of tokens per frame, errors compound over the rollout, and most of every predicted frame restates what the observed history already shows: the scene layout, the identity and appearance of objects, and everything the action does not touch.

The opposite extreme is to skip prediction altogether and learn a *direct scorer* that maps the history, a candidate action, and a goal to a score [30]. A direct scorer is cheap, but it entangles two different things: what the action will do, which does not depend on the goal, and how the outcome relates to the goal, which does. It must therefore learn the consequences of actions separately for every goal, and it has no reason to generalize to goals, or to questions, that it was not trained on.

This paper asks whether a world model can occupy the space between these extremes: *instead of simulating the future frame by frame, can a world model predict a compact abstraction of an entire future segment that is still sufficient for decision making?* Two observations make this plausible. First, a planner never needs the future as such, only the answers to a few questions about it: how much closer to the goal the segment ends, whether it passes through a state resembling a reference image, or whether one event happens before another. Second, whatever the observed history already determines need not be predicted again. Learned video compression exploits the same fact through *conditional coding*, where both the encoder and the decoder see the temporal context and bits are spent only on what the context cannot predict [25]. We apply this principle to the *unexecuted* future of an action.

We propose *Conditional Trajectory Abstraction* (CTA), summarized in Fig. 1. A cached context C encodes the observed history once per decision. During training only, a source encoder compresses the actual future segment of an action chunk into M discrete tokens S conditioned on

036
037
038
039
040
041
042
043
044
045
046
047
048
049
050
051
052
053
054
055
056
057
058
059
060
061
062
063
064
065
066
067
068
069
070
071
072
073
074
075

C . A reader that receives C , S , and a query answers self-supervised visual-temporal questions about the segment, whose ground truth is computed from frozen features of the actual future. The reader never receives the action, so everything the action changes must pass through S . The world model is an action-conditioned predictor of S given C and a candidate chunk. Because a code is useful only if it can be forecast, the source code is *co-designed* with the world model: it is trained to retain query answers at a bounded rate while remaining predictable from the action. In policy-guided planning, a frozen pretrained policy proposes a small bank of candidate chunks, the world model predicts one code per candidate, and the reader scores each code against the goal. Since codes do not depend on the goal, one prediction per candidate serves any number of goals and queries: *predict once, query many*.

Each ingredient of CTA has precedents in world models, video compression, and policy steering, so a new name is not a contribution by itself. Our claims are therefore stated as comparisons at matched history, proposal bank, and compute, against the two extremes above and against controls that remove one ingredient at a time. Our contributions are as follows.

- We formulate trajectory abstraction for world models as *conditional* rate-distortion over a family of self-supervised visual-temporal queries, subject to predictability from actions, and show how frame-level world models and direct scorers arise as its two limiting cases.
- We introduce CTA, a world model that predicts a history-conditioned, segment-level discrete code of an action's future, together with a query reader and a co-design objective that trades query distortion, code rate, and action predictability. No component uses rewards, success labels, or simulator state.
- We define an evaluation protocol that first qualifies the planning interface, namely whether the proposal bank contains better candidates and whether the query-based score selects them when given actual futures, and then isolates each claim with a matched control: per-frame codes at equal total bits, a reader without history, a code trained without the predictability term, a direct scorer on held-out goals, and a frame-level world model at matched compute.
- On PushT [8, 14] and LIBERO-Goal [26] with frozen released policies, [result placeholder].

2. Related Work

World models for visual planning. Latent world models learn dynamics in a compact state space and plan or learn behaviors in imagination [17–19]. Token-based models cast dynamics as autoregressive sequence modeling over discrete image tokens [28], and Δ -IRIS reduces their cost by encoding only the stochastic change between consecu-

tive frames given the past [29]. DINO-WM [38] and V-JEPA 2-AC [3] predict in the patch-feature space of a pretrained encoder [5, 31] and plan by minimizing the feature distance between the predicted final frame and a goal image. CompACT [22] compresses each observation into as few as eight discrete tokens to accelerate such planning, and GCQ [32] compresses observation-action sequences with an action-conditioned codebook. Almost all of these models are trained to reproduce observations, and they are queried by rolling out one step per frame. CTA differs in its unit of prediction and in its training signal. It predicts one code per action segment, conditioned on the history available to the reader, and the code is shaped by the questions a planner asks and by its predictability from actions rather than by reconstruction of observations. Compact per-frame codes are therefore our most important control: a per-frame code with the same context and the same total number of bits must not match CTA if the segment-level claim is to hold.

Temporally extended prediction. Option models [35] and multi-step or trajectory-level dynamics models [10, 24] predict the outcome of a temporally extended action without stepping through every intermediate state, and jumpy world models compose such predictions across timescales [13]. Successor features and their random-feature variants predict cumulative features of a policy or an action sequence [6, 7], and TD-Flow learns generative models of discounted future-state occupancy [12]. These representations summarize the future either as states at a given time or as order-free aggregates. CTA instead retains ordered, non-additive information about a segment, such as whether one event precedes another, and it conditions the summary on the observed history rather than predicting it from scratch.

Policy steering with world models. A frozen generative policy can be improved at deployment by sampling several action proposals and selecting one with an external scorer. GPC [33] ranks diffusion-policy proposals by task reward predicted on the rollouts of an action-conditioned world model. FOREWARN [36] aligns a vision-language model to latent predictions of candidate plans. DreamSteer [9] ranks imagined latent futures of VLA chunks with a language-conditioned value model, and DynaGuide [11] steers denoising with a DINOv2-space dynamics model. Value- or verifier-based methods rank proposals without a world model [23, 30]. We adopt the same propose-then-select interface, with a frozen policy and a shared proposal bank for every compared model, but our contribution concerns what the world model predicts. Our scores come from self-supervised queries on predicted codes rather than from rewards, success labels, or a vision-language model.

Conditional coding and decision-aware compression. Learned image and video codecs optimize a rate-distortion trade-off end to end [4], and DCVC [25] replaces residual coding with conditional coding, in which both encoder and

Deployment: predict once, query many

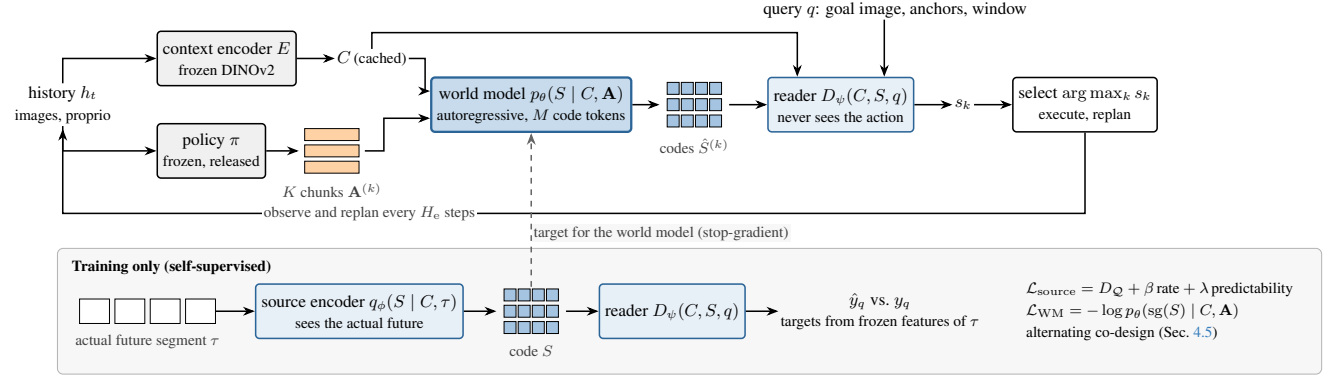


Figure 1. **Conditional Trajectory Abstraction.** *Top:* at each decision, the history is encoded once into a context C , and a frozen policy proposes K action chunks. The world model predicts, for each chunk, a short discrete code $\hat{S}$ of the chunk’s entire future segment, conditioned on C . A reader that sees C and the code, but never the action, answers queries such as the remaining steps to a goal image. Since codes do not depend on the query, one prediction per candidate serves any number of goals. *Bottom:* during training, a source encoder compresses the actual future segment into the code, the reader is trained on self-supervised query targets computed from frozen features of that future, and the world model learns to predict the code; code and world model are co-designed for query retention, rate, and predictability.

decoder use the temporal context. Unlike the Wyner–Ziv setting [37], where side information is available only to the decoder, our encoder and reader both see the context. In reinforcement learning, value-equivalent models [16, 34] retain only what is needed to predict values, and value-equivalent sampling [2] formalizes decision-relevant lossy model compression. CTA is an instance of these principles with two differences. Its source is the unexecuted future of an action, which must also be predicted before execution; predictability is therefore an explicit term of the objective. In addition, its distortion is measured on an open family of self-supervised visual–temporal queries rather than on the values of a fixed set of policies.

3. Problem Formulation

Setting. A robot observes images o_t and proprioceptive states q_t . A frozen visual encoder φ maps each image to a grid of patch features, and a context $C_t = E(h_t)$ summarizes the observed history h_t using only present and past information. At a decision time t , a candidate action chunk $\mathbf{A} = (a_t, \dots, a_{t+H-1})$ is executed for its first H_e steps. The *future segment* of $\mathbf{A}$ is $\tau = (\varphi(o_{t+1}), \dots, \varphi(o_{t+H_e}))$, the frozen features of the observations that executing $\mathbf{A}$ produces, at the native temporal resolution. A world model must say something about τ *before* $\mathbf{A}$ is executed.

Queries. What a planner needs to know about τ is expressed as a family $\mathcal{Q}$ of queries. A query q has a type and parameters, such as a goal image, one or two anchor images, or a time window. Its answer y_q is computed from frozen features of the actual future, so no annotation is required. We use five types: (i) the similarity of the segment’s end to a goal image; (ii) the maximum similarity to an anchor

image over the segment; (iii) the number of steps from the end of the segment to a goal frame later in the same episode; (iv) whether anchor u is matched before anchor v within the segment; and (v) masked local probes of spatial features, which retain fine motion. Types (ii) and (iv) depend on the path and not only on the endpoint. Type (iii) measures temporal progress rather than visual similarity, which matters in multi-stage tasks where the distance to a goal image is a poor measure of how far along the task is.

Conditional trajectory abstraction. A *conditional trajectory code* is a sequence of M discrete tokens $S \in \{1, \dots, V\}^M$ produced by a source encoder $q_\phi(S | C, \tau)$, which sees the actual future and is used only during training. A reader $D_\psi(C, S, q)$ answers queries from the context and the code, and a world model $p_\theta(S | C, \mathbf{A})$ predicts the code from the context and the candidate chunk. The reader never receives $\mathbf{A}$, so any information about the action’s consequences must pass through S . In its idealized form, the code solves a conditional rate–distortion problem,

$$\min_{q_\phi, D_\psi} \mathbb{E}_{q \sim \mathcal{Q}} \mathbb{E}[d(D_\psi(C, S, q), y_q)] \quad \text{s.t.} \quad I(\tau; S | C) \leq R, \quad (1)$$

in which the context is available to both the encoder and the reader. We count rate by the code itself: $M \in \{8, 16, 32\}$ tokens with vocabulary $V=256$ correspond to at most 64, 128, and 256 bits per segment. This count excludes the codebook, the reader, and the context cache, which we report separately.

Predictability. At deployment the code is not observed but predicted, $\hat{S} \sim p_\theta(\cdot | C, \mathbf{A})$, so what matters is the *deployment* distortion $\mathbb{E} d(D_\psi(C, \hat{S}, q), y_q)$. A code that minimizes (1) alone may spend its rate on aspects of τ that

	Predicted per candidate	Sequential steps	New goal or query	Path queries
Frame-level WM	$H_e \cdot P$ tokens	H_e	reader only	✓
Endpoint latent	P tokens	1	reader only	–
Direct scorer	one score	1 per query	new evaluation	trained only
CTA (ours)	M tokens	M	reader only	✓

Table 1. **Design properties** (not results) of world models for evaluating a candidate chunk. P is the number of patch tokens per frame (hundreds), $M \in \{8, 16, 32\}$ the number of code tokens. CTA’s M autoregressive steps are short but not automatically cheaper than H_e frame steps, so latency is measured, not assumed.

the action does not determine, and those aspects cannot be forecast. We therefore also require the code to be predictable, $\mathbb{E}[-\log p_\theta(S \mid C, \mathbf{A})]$ small, and design the code and the world model jointly (Sec. 4.5).

Two limiting cases. The formulation contains the two common designs as extremes. A *frame-level world model* sets $S = \tau$: it has no codec distortion and unbounded rate, and it must forecast H_e frames of patch features per candidate. A *direct scorer* $D(C, \mathbf{A}, q)$ removes S altogether: it has zero rate, but prediction and reading are fused, so each candidate–query pair requires a separate evaluation and generalization to new queries must be learned jointly with the effect of the action. CTA operates at a finite, explicit rate between them. The questions we study are whether, at such a rate, predicted codes answer queries as accurately as a frame-level model at lower cost, and whether they generalize to held-out goals better than a direct scorer. Table 1 summarizes the design properties.

Policy-guided planning. We evaluate world models in the planning interface of recent policy steering work [9, 33]. At every decision, a frozen pretrained policy π proposes K chunks $\mathbf{A}^{(1)}, \dots, \mathbf{A}^{(K)}$ from independent noise seeds, where $\mathbf{A}^{(1)}$ is the chunk the unmodified policy would execute. Each world model scores every proposal with the same goal query, the highest-scoring proposal is executed for H_e steps with ties resolved in favor of $\mathbf{A}^{(1)}$, and the policy replans. Selecting among proposals of a competent policy, rather than optimizing actions against the model, keeps candidates on the policy’s support, where a learned model is less exposed to adversarial exploitation of its errors [15, 21]. The same proposal bank is given to every compared model. Because no world model can recover an action that is absent from the bank, our evaluation first verifies that the bank contains better candidates (Sec. 5).

4. Conditional Trajectory Abstraction

CTA consists of a context encoder (Sec. 4.1), a source encoder used only in training (Sec. 4.2), a query reader (Sec. 4.3), and the world model that predicts codes from actions (Sec. 4.4). Section 4.5 describes how the code and the

world model are trained jointly, Sec. 4.6 how they are used for planning, and Sec. 4.7 which control tests each design choice.

4.1. Context encoder

Images are embedded by a frozen DINOv2 ViT-S/14 [31], following DINO-WM [38], and the full patch grid is kept. A small Transformer E maps the patch features of the most recent n frames and the corresponding proprioceptive states to a set of context tokens C_t . The context is computed once per decision and shared by all K candidates and all queries; its cost is reported once and not multiplied by K . Every compared model receives the same history.

4.2. Source encoder

The source encoder $q_\phi(S \mid C, \tau)$ turns the actual future segment into a code and exists only during training. The patch features of the H_e future frames receive spatial and native-time position embeddings. M learned segment queries cross-attend jointly to these future tokens and to the context tokens, and each output is projected and quantized by finite scalar quantization [27] with an implicit vocabulary of $V=256$. Because the segment queries attend to the context, the encoder can learn which parts of the future are already implied by the history instead of relying on a hand-designed residual such as subtracting current from future features. The code S describes the whole segment at a declared temporal resolution; it is not a set of M tokens per frame.

4.3. Query reader

The reader $D_\psi(C, S, q)$ answers a query from the context and the code. A query encoder maps the query type to a learned embedding and its parameters to tokens: goal and anchor images through φ , and time windows through a learned embedding. The query tokens cross-attend to the context tokens and the embedded code tokens over several layers, and a type-specific head produces the answer. The reader has no access to the action, so the code is the only channel through which the action’s consequences reach it. We use squared error for the continuous query types and logistic loss for the order query.

Self-supervised query targets. All targets are computed from the frozen feature space of the actual future, using fixed feature kernels and normalization statistics estimated on training episodes only. For the progress query (iii), the goal is a frame sampled from later in the same episode and the target is $\log(1 + \Delta t)$, where Δt is the number of steps from the end of the segment to that frame. This is a form of hindsight relabeling [1, 20] that turns every episode, successful or not, into supervision. For the similarity and order queries, anchors are sampled from the data together with hard negatives. Anchors, goal sets, and windows used for evaluation are fixed independently of the candidate being

scored, the query family and its weights are frozen before any planning evaluation, and episodes are split by root trajectory before any query is constructed. No reward, success label, simulator state, contact label, or human annotation enters the training of the context encoder, the source encoder, the reader, or the world model. The frozen proposal policy is a separately pretrained imitation-learning model, so the complete system is not free of expert data, and we state this explicitly.

4.4. Action-conditioned world model

The world model $p_\theta(S | C, \mathbf{A})$ predicts the code of a candidate’s future before it is executed. Each action of the executed prefix $\mathbf{A}_{1:H_e}$ is embedded linearly with a temporal embedding. A Transformer attends over the context tokens and the action tokens and decodes the M code tokens autoregressively as categorical distributions over the vocabulary, which preserves the dependencies between code positions. The world model does not receive future images, future proprioception, the query, or any label. It is trained with teacher forcing on source codes. At test time we decode either the most likely code or a small number of sampled codes; with samples, we average the reader’s *answers* rather than the code embeddings, since the reader is nonlinear.

4.5. Co-designing the code and the world model

Training alternates between the code and the world model in three stages. Let $D_Q = \mathbb{E}_{q \sim \mathcal{Q}} d(D_\psi(C, S, q), y_q)$ denote the query distortion with $S \sim q_\phi(\cdot | C, \tau)$, and let $r_\omega(S | C)$ be a context-only autoregressive model of the code that serves as a rate estimate.

Stage 1: codec warm-up. The source encoder, the reader, and the rate model are trained on actual futures and sampled queries by minimizing $D_Q + \beta \mathbb{E}[-\log r_\omega(S | C)]$.

Stage 2: world model. The world model is trained to predict the stop-gradient source codes,

$$\mathcal{L}_{\text{WM}} = \mathbb{E}[-\log p_\theta(\text{sg}(S) | C, \mathbf{A})]. \quad (2)$$

Stage 3: alternating co-design. A fixed, predeclared number of source updates alternates with a fixed number of world-model updates. The source objective adds a predictability term under a frozen copy $\bar{p}_\theta$ of the world model,

$$\mathcal{L}_{\text{source}} = D_Q + \beta \mathbb{E}[-\log r_\omega(S | C)] + \lambda \mathbb{E}[-\log \bar{p}_\theta(S | C, \mathbf{A})], \quad (3)$$

which favors codes that the world model can forecast from the action. Within each world-model block, the targets are fixed codes from an exponential moving average of the source encoder. Gradients pass through the quantizer with the straight-through estimator, which is an estimator rather than an exact gradient. Setting $\lambda=0$ recovers a code trained

Algorithm 1 Policy-guided planning with CTA.

Require: frozen policy π ; context encoder E ; world model p_θ ; reader D_ψ ; goal images $\mathcal{G}$; K ; H_e

```

1: while episode not finished do
2:    $C \leftarrow E(h_t)$  ▷ once per decision
3:    $\mathbf{A}^{(k)} \leftarrow \pi(h_t; \xi_k)$  for  $k = 1, \dots, K$  ▷ shared bank
4:    $\hat{S}^{(k)} \leftarrow \text{decode } p_\theta(\cdot | C, \mathbf{A}_{1:H_e}^{(k)})$  for all  $k$  ▷ batched
5:    $s_k \leftarrow -\frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} D_\psi(C, \hat{S}^{(k)}, q_{\text{prog}}(g))$ 
6:    $k^* \leftarrow \text{smallest index attaining } \max_k s_k$  ▷ ties keep the default
7:   execute  $\mathbf{A}_{1:H_e}^{(k^*)}$ ;  $t \leftarrow t + H_e$ 

```

for query retention and rate only, and this ablation tests whether predictability-aware co-design adds anything.

Training data and collapse diagnostics. Training tuples $(h_t, \mathbf{A}, \tau, \text{continuation})$ come from demonstrations and from rollouts of the proposal policy, including several chunks branched from the same simulator state so that the world model observes different actions from identical histories. All branches of a root episode belong to the same data split. A code can satisfy the losses while ignoring the action. We therefore monitor code usage, the query accuracy of a history-only baseline that receives no code, and whether codes differ between chunks proposed from the same state.

4.6. Planning with predicted codes

Algorithm 1 summarizes deployment. The planning score of a candidate is the reader’s answer to the progress query (iii) for the task’s goal images, averaged over the goal images and negated, so that the candidate predicted to leave the fewest remaining steps is selected. At deployment, the goal images are final frames of successful demonstrations of the task. The query type used for planning is fixed before evaluation, and other types serve as training signal and as held-out evaluations of the code. Per decision, CTA encodes the context once, runs the world model on K candidates in one batch, and runs the reader on $K|\mathcal{G}|$ candidate–goal pairs. Because codes do not depend on the goal, a new goal or query on the same candidates costs only additional reader evaluations.

4.7. Design choices and the controls that test them

Each design choice is a hypothesis with a matched control, evaluated at the same history, proposal bank, data, and training budget.

A segment code rather than per-frame codes. A decision is made once per chunk and the chunk is executed open loop, so no intermediate state is ever selected. Some queries nevertheless depend on the path, not only on its endpoint. A segment code can allocate its bits across the segment where they matter, whereas a per-frame code must spend them uniformly. Because the source encoder sees the entire

segment, a brief event such as the onset of contact is present in the target whenever it changes a query’s answer. *Controls*: a conditional per-frame code with the same context and the same total number of bits, a code of the endpoint only, and uniform temporal pooling of per-frame features. If the per-frame code matches CTA, the segment claim fails.

Conditioning on the history. The reader and the source encoder both see C , so the code need not restate the scene. *Control*: a reader without C , which forces scene information through the same bottleneck.

Predictability-aware co-design. A code that is informative but not forecastable from the action is of no use to a world model. *Control*: $\lambda=0$ in (3).

Predicting a code rather than a score. A direct scorer $D(C, A, q)$ is the minimal model sufficient for the goals it was trained on. The advantage of factoring prediction from reading should appear on goals and queries that were not seen during training. *Control*: a direct scorer with the same data and capacity, evaluated on held-out goals of LIBERO-Goal, whose ten tasks share one scene.

Predicting a code rather than frames. A frame-level world model answers any query but pays for every frame. *Control*: a DINO-WM-style patch-feature world model trained on the same data, with the same queries computed on its predicted frames, compared at matched compute and on the full accuracy–latency frontier over $M \in \{8, 16, 32\}$. We also compare against GPC [33] as a policy-steering system, against a compact per-frame tokenizer in the style of CompACT [22], and against the sequence quantization of GCQ [32] where its released mechanism applies to our interface.

5. Experiments

[TODO: Not yet written. Planned structure, following the pre-registered gate sequence: (1) arenas: PushT with the released Diffusion Policy (pipeline and interface qualification; single fixed goal), LIBERO-Goal with released SmolVLA / π_0 chunk policies (main arena; ten goals in one scene, held-out goals); (2) interface qualification: policy reproduction, proposal-bank headroom under a physical reference score, and whether the query-based score selects well on *actual* futures; (3) codec on actual futures: query distortion vs. bits for the segment code, the per-frame code at equal total bits, the endpoint code, and a reader without history, including held-out goals and queries; (4) full world model in closed loop: CTA vs. policy only, a direct scorer, a DINO-WM-style frame world model, and GPC, on the same proposal bank; accuracy–latency frontier over $M \in \{8, 16, 32\}$; (5) ablations: $\lambda=0$, uniform temporal pooling, MAP vs. sampled codes; (6) confirmation on fresh episodes with three training seeds and episode-clustered confidence intervals.]

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 4
- [2] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 3
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [4] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017. 2
- [5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research (TMLR)*, 2024. 2
- [6] André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2
- [7] Boyuan Chen, Chuning Zhu, Pulkit Agrawal, Kaiqing Zhang, and Abhishek Gupta. Self-supervised reinforcement learning that transfers using random features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [8] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. 2
- [9] Hanchen Cui et al. DreamSteer: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 2, 4
- [10] Zihan Ding, Amy Zhang, Yuandong Tian, and Qinqing Zheng. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024. 2
- [11] Maximilian Du and Shuran Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [12] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows. In *International Conference on Machine Learning (ICML)*, 2025. 2
- [13] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Marc G. Bellemare, Alessandro Lazaric, and Ahmed Touati. Compositional planning with jumpy world models. *arXiv preprint arXiv:2602.19634*, 2026. 2
- [14] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzan Wahid, Laura Downs, Adrian Wong, Johnny Lee,

- 521 Igor Mordatch, and Jonathan Tompson. Implicit behavioral
522 cloning. In *Conference on Robot Learning (CoRL)*, 2021. 2
- 523 [15] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for
524 reward model overoptimization. In *International Conference*
525 *on Machine Learning (ICML)*, 2023. 4
- 526 [16] Christopher Grimm, André Barreto, Satinder Singh, and
527 David Silver. The value equivalence principle for model-
528 based reinforcement learning. In *Advances in Neural Infor-*
529 *mation Processing Systems (NeurIPS)*, 2020. 3
- 530 [17] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Ville-
531 gas, David Ha, Honglak Lee, and James Davidson. Learning
532 latent dynamics for planning from pixels. In *International*
533 *Conference on Machine Learning (ICML)*, 2019. 1, 2
- 534 [18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lil-
535 licrap. Mastering diverse control tasks through world models.
536 *Nature*, 2025.
- 537 [19] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2:
538 Scalable, robust world models for continuous control. In *In-*
539 *ternational Conference on Learning Representations (ICLR)*,
540 2024. 2
- 541 [20] Kristian Hartikainen, Xinyang Geng, Tuomas Haarnoja,
542 and Sergey Levine. Dynamical distance learning for semi-
543 supervised and unsupervised skill discovery. In *International*
544 *Conference on Learning Representations (ICLR)*, 2020. 4
- 545 [21] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine.
546 When to trust your model: Model-based policy optimiza-
547 tion. In *Advances in Neural Information Processing Systems*
548 *(NeurIPS)*, 2019. 4
- 549 [22] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and
550 Suha Kwak. Planning in 8 tokens: A compact discrete tok-
551 enizer for latent world model. In *IEEE/CVF Conference on*
552 *Computer Vision and Pattern Recognition (CVPR)*, 2026. 2, 6
- 553 [23] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter,
554 Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone.
555 RoboMonkey: Scaling test-time sampling and verification
556 for vision-language-action models. In *Conference on Robot*
557 *Learning (CoRL)*, 2025. 2
- 558 [24] Nathan O. Lambert, Albert Wilcox, Howard Zhang, Kristofer
559 S. J. Pister, and Roberto Calandra. Learning accurate long-
560 term dynamics for model-based reinforcement learning. In
561 *IEEE Conference on Decision and Control (CDC)*, 2021. 2
- 562 [25] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video com-
563 pression. In *Advances in Neural Information Processing*
564 *Systems (NeurIPS)*, 2021. 1, 2
- 565 [26] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu,
566 Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowl-
567 edge transfer for lifelong robot learning. In *Advances in*
568 *Neural Information Processing Systems (NeurIPS), Datasets*
569 *and Benchmarks Track*, 2023. 2
- 570 [27] Fabian Mentzer, David Minnen, Eirikur Agustsson, and
571 Michael Tschannen. Finite scalar quantization: VQ-VAE
572 made simple. In *International Conference on Learning Rep-*
573 *resentations (ICLR)*, 2024. 4
- 574 [28] Vincent Micheli, Eloi Alonso, and François Fleuret. Trans-
575 formers are sample-efficient world models. In *International*
576 *Conference on Learning Representations (ICLR)*, 2023. 1, 2
- 577 [29] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient
578 world models with context-aware tokenization. In *Interna-*
579 *tional Conference on Machine Learning (ICML)*, 2024. 2
- 580 [30] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey
581 Levine. Steering your generalists: Improving robotic foun-
582 dation models via value guidance. In *Conference on Robot*
583 *Learning (CoRL)*, 2024. 1, 2
- 584 [31] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy
585 Vo, et al. DINOv2: Learning robust visual features without
586 supervision. *Transactions on Machine Learning Research*
587 *(TMLR)*, 2024. 2, 4
- 588 [32] Xiangyuan Peng, Xingsi Dong, and Si Wu. Vector quanti-
589 zation in the brain: Grid-like codes in world models. In *Ad-*
590 *vances in Neural Information Processing Systems (NeurIPS)*,
591 2025. 2, 6
- 592 [33] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang.
593 Inference-time enhancement of generative robot policies via
594 predictive world modeling. *IEEE Robotics and Automation*
595 *Letters (RA-L)*, 2025. 2, 4, 6
- 596 [34] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert,
597 Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez,
598 Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy
599 Lillicrap, and David Silver. Mastering Atari, Go, chess and
600 shogi by planning with a learned model. *Nature*, 588:604–
601 609, 2020. 3
- 602 [35] Richard S. Sutton, Doina Precup, and Satinder Singh. Be-
603 tween MDPs and semi-MDPs: A framework for temporal
604 abstraction in reinforcement learning. *Artificial Intelligence*,
605 112(1–2):181–211, 1999. 2
- 606 [36] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From
607 foresight to forethought: VLM-in-the-loop policy steering via
608 latent alignment. In *Robotics: Science and Systems (RSS)*,
609 2025. 2
- 610 [37] Aaron D. Wyner and Jacob Ziv. The rate-distortion function
611 for source coding with side information at the decoder. *IEEE*
612 *Transactions on Information Theory*, 22(1):1–10, 1976. 3
- 613 [38] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto.
614 DINO-WM: World models on pre-trained visual features en-
615 able zero-shot planning. In *International Conference on Ma-*
616 *chine Learning (ICML)*, 2025. 1, 2, 4